

# Gen AI — Phase 1 Complete Notes (Interview-Ready)

Simple, easy-English notes for quick revision before interviews. Covers: Tokenization, Embeddings, Attention, Transformer Architecture, Text Generation/Sampling.

## 1. Tokenization

**What it is:** Before an AI model reads text, it breaks it into small pieces called **tokens**. A token can be a full word, part of a word, or a punctuation mark.

Example: `"unbelievable"` → `["un", "believ", "able"]`

**Why not use whole words?** Too many possible words (every language, every misspelling) would make the vocabulary huge and hard to manage. Breaking into smaller reusable pieces (subwords) solves this — even new/rare words can be built from familiar pieces.

**Method used:** BPE (Byte Pair Encoding) — learns common chunks of text from huge datasets.

**Why it matters:** API pricing is based on tokens, not words. Context window limits (like "128k tokens") are also measured in tokens.

## 2. Embeddings

**What it is:** Once text is split into tokens, each token is converted into a list of numbers (a **vector**) that represents its *meaning*.

**Simple analogy:** Think of each word getting "meaning sliders" (like a character creator). Words with similar meaning get similar numbers.

**Famous example:** `king - man + woman ≈ queen` — this works because embeddings capture *relationships* between words, not just individual meanings.

**How measured:** Similarity between two embeddings is calculated using **cosine similarity** (closer to 1 = more similar, closer to 0 = unrelated).

**Why it matters:** This is the foundation of semantic search and RAG (Retrieval Augmented Generation) — finding relevant info based on meaning, not exact keyword match.

## 3. Attention

**What it is:** A mechanism that lets the model look at **all words in a sentence at once**, and decide which words are most relevant to each other.

**Simple analogy (library search):**

Query (Q) = what you're searching for

Key (K) = the label on each item

Value (V) = the actual content you get if it matches

The model compares each word's Query against every other word's Key to find the best matches, then blends in their Values.

**Example:** In *"The cat sat on the mat because it was tired,"* attention helps the model figure out "it" = "cat," not "mat."

**Self-Attention:** Words attend to other words within the same sentence.

**Multi-Head Attention:** The model does this attention process multiple times in parallel (different "heads"), each focusing on a different type of relationship (grammar, meaning, references, etc.), then combines the results.

**Why it matters:** This is the core innovation behind Transformers (the "T" in GPT) — it lets models understand long-range context, unlike older models that read word-by-word in order.

## 4. Transformer Architecture

**What it is:** The overall structure/design of modern LLMs. Made of repeated blocks, each containing:  
Multi-Head Attention (relates words to each other)

Feed-Forward Network (processes each word further)

Stacked many times (12, 32, 96+ layers) — more layers = more powerful model.

**Three types:**

**Encoder-only** (e.g. BERT) — good for understanding tasks (classification, search), not generation

**Decoder-only** (e.g. GPT, Claude, Grok) — generates text one token at a time; used by almost all modern chatbots

**Encoder-Decoder** (e.g. original Transformer, T5) — used for tasks like translation

**Why decoder-only for chatbots:** Since text is generated step-by-step, the model should only see previous words, not future ones (which don't exist yet). This rule is called **causal/masked attention**.

**Other supporting parts:**

**Positional Encoding** — tells the model the order of words (since attention doesn't naturally know order)

**Layer Normalization** — keeps numbers stable during training

**Residual Connections** — shortcuts that help information flow through deep layers without getting lost

## 5. Text Generation / Sampling

**What it is:** How the model actually picks the next word. The model doesn't give one fixed answer — it gives a **probability list** for every possible next word, then a strategy picks one.

**Key methods:**

**Greedy decoding** — always pick the highest probability word (safe but repetitive)

**Temperature** — controls randomness. Low (0.1-0.3) = safe/predictable. High (0.7-1.5) = more creative/random. Temperature 0 ≈ greedy.

**Top-k sampling** — only consider the top K most likely words, pick from those

**Top-p (nucleus) sampling** — keep adding words until their combined probability crosses a threshold (like 90%), pick from that group

**Practical use:**

Factual tasks (support bot, code) → low temperature

Creative tasks (story writing, brainstorming) → higher temperature

**Important nuance:** Temperature's effect also depends on the prompt. Factual prompts naturally have less room for variation, so even high temperature might not change output much. Temperature shows bigger effect on open-ended/creative prompts.

## Full Pipeline (How It All Connects)



Raw Text

- Tokenize (break into tokens)
- Embed (convert tokens into meaning vectors) + Positional Encoding (add order info)
- Pass through Transformer layers (Attention + Feed-Forward, repeated many times)
- Output probability distribution over all possible next tokens
- Sampling strategy (temperature/top-p) picks the next token
- Repeat until response is complete

## Most Important Interview Q&A (Short & Simple Answers)

**Q1. What is a token?** A: A small piece of text (word, part of word, or symbol) that the model processes — not the same as a full word.

**Q2. Why do LLMs use subword tokenization instead of whole words?** A: To keep vocabulary manageable and handle rare/new words by breaking them into familiar smaller pieces.

**Q3. What is an embedding?** A: A list of numbers (vector) that represents a word's meaning, allowing the model to do math on language.

**Q4. What does cosine similarity measure?** A: How similar two embeddings (vectors) are — value closer to 1 means more similar meaning, closer to 0 means unrelated.

**Q5. Explain "king - man + woman = queen."** A: It shows embeddings capture relationships between words, not just individual meaning — vector math on embeddings reflects real-world analogies.

**Q6. What problem does attention solve?** A: It lets the model connect relevant words regardless of distance in a sentence, unlike older models that struggled with long-range context.

**Q7. What are Query, Key, and Value in attention?** A: Query = what a word is looking for. Key = what each word offers. Value = the actual info passed along if there's a match.

**Q8. What is multi-head attention?** A: Running attention multiple times in parallel, each head focusing on a different type of relationship (grammar, meaning, etc.), then combining results.

**Q9. What's the difference between encoder-only, decoder-only, and encoder-decoder models?** A: Encoder-only understands text (e.g. BERT). Decoder-only generates text (e.g. GPT/Claude). Encoder-decoder does both, mainly for tasks like translation.

**Q10. Why do GPT-style models use causal/masked attention?** A: Because they generate text step-by-step and should only see previous words, not future ones that don't exist yet.

**Q11. What is positional encoding and why is it needed?** A: It tells the model the order of words, since attention alone doesn't know word sequence.

**Q12. What is temperature in text generation?** A: A setting that controls randomness — low temperature = safe/predictable output, high temperature = more creative/random output.

**Q13. What's the difference between top-k and top-p sampling?** A: Top-k picks from a fixed number of top choices. Top-p picks from a dynamic group of choices whose combined probability crosses a set threshold (like 90%).

**Q14. What is a context window?** A: The total number of tokens (input + output combined) a model can process in one go. Anything beyond this limit gets cut off.

**Q15. Does context window mean the model remembers you across different chats?** A: No — each new conversation starts fresh. Apps that seem to "remember" you are separately storing and re-adding past

info; that's not the context window itself.

**Q16. Why does token count differ across languages for the same sentence?** A: Tokenizers are trained on how much text of each language exists online. Well-represented languages (like English) get efficient whole-word tokens; less-represented languages get split into more, smaller pieces.

**Q17. What's the full pipeline from text input to AI output?** A: Text → Tokens → Embeddings → Transformer layers (Attention + Feed-Forward) → Probability distribution → Sampling → Next token → repeat.

*End of Phase 1 notes. Next: Phase 2 — Prompt Engineering.*